Base44 → Own Stack Migration Plan

Project: Sprint Capacity Planning **Date:** March 2026 **Status:** Planning — not yet started

---

1. Executive Summary

The application currently runs on **Base44**, a proprietary Backend-as-a-Service (BaaS) platform that provides the database, authentication, user management, and server-side function runtime. Moving to an owned stack removes the vendor dependency and gives full control over data, infrastructure, and cost.

The recommended migration target is **Supabase** — an open-source BaaS built on PostgreSQL — because it is the closest architectural match and minimises rewrite risk. Supabase can also be self-hosted if that is required in the future.

The migration is **low-risk to the frontend** because the UI logic, React component tree, routing, and all business logic are already decoupled from Base44 through a thin API client layer. The core work is replacing that client layer and the nine server-side functions.

---

2. What Base44 Currently Provides

Layer	Role in the Application	Files Affected
Database / ORM	13 entity tables; all reads, writes, and deletes via `base44.entities.X.list / create / update / delete`	26 source files
Authentication	Login, logout, session management, user invitation via `base44.auth.*`	4 core files
Server-side functions	9 Deno functions: Jira sync, user invite, cleanup tasks	9 files in `/base44/functions/`
User management	Role model (admin / team_manager / viewer), managed team IDs	`permissions.js`, `UserManagement`
Vite build plugin	`@base44/vite-plugin` — build-time integration	`vite.config.js`

---

3. Entity Inventory

All 13 entities that must be migrated to database tables:

Entity	Purpose	Operations Used
`Team`	Team definitions	CRUD
`TeamMember`	Members assigned to teams, with discipline & availability	CRUD
`User`	Platform users with roles and team assignments	CRUD
`Sprint`	Sprint containers (team-specific or cross-team templates)	CRUD
`WorkArea`	Work items / epics with Jira linkage	CRUD
`WorkAreaType`	Classification categories for work areas	CRUD
`Allocation`	Sprint-level capacity allocation (member × sprint × work area)	CRUD
`QuarterlyAllocation`	Quarterly capacity allocation (member × quarter × work area)	CRUD
`QuarterlyWorkAreaSelection`	Work areas selected by a team for a given quarter	CRUD
`QuarterlyPlanHistory`	Audit log of quarterly plan changes	Create, Read
`JiraSyncHistory`	Record of Jira sync operations	Read
`PendingUserTeams`	Temporary storage for pending user team assignments (back-end only)	Create, Read, Delete

4. Server-Side Functions Inventory

Nine Deno functions currently running inside Base44:

Function	Purpose	External Dependencies
`ensureUserExists`	Creates a `User` record on first login	None
`applyPendingTeams`	Applies pending role / team assignments after invite acceptance	None
`inviteUserWithTeams`	Sends email invitation + stores pending team assignments	Base44 user invite API
`jiraSync`	Fetches issues from Jira, extracts teams, types, work areas	Jira REST API v3

`syncJiraIssues`	Updates existing work areas with current Jira status and progress	Jira REST API v3
`linkJiraEpic`	Links a Jira epic key to a work area record	Jira REST API v3
`validateUserAccess`	Checks whether a user is registered (currently unused in UI)	None
`cleanupOrphanedAllocations`	Deletes allocations with missing member or work area references	None
`cleanupAllOrphans`	Comprehensive cleanup of all orphaned records across entities	None

5. Recommended Migration Target: Supabase

Why Supabase

Requirement	How Supabase Covers It
Relational database	PostgreSQL — same SQL semantics, foreign keys, indexes
CRUD API	Auto-generated REST API and JS client — mirrors `base44.entities.X` pattern
Authentication	Supabase Auth: email/password, magic link, OAuth, user invitations built in
Server-side functions	Edge Functions running on **Deno** — the Jira sync functions port with minimal changes
Self-hosting option	Fully open source; can run on your own infrastructure via Docker
Role-based access	Row Level Security (RLS) policies, or keep permission checks in the app layer as-is

What Stays the Same

Everything outside the API client layer requires **zero changes**:

- All React components and pages
- React Query caching and data-fetching patterns
- `permissions.js` role logic
- Routing (React Router v6)
- UI (Radix UI + Tailwind)
- All export and reporting functionality
- Charts and visualisations

6. Migration Phases

Phase 1 — Database Schema & Data Layer

Estimated effort: 1–2 weeks

1. Define all 13 tables as a PostgreSQL schema in Supabase (columns, types, foreign keys, indexes)
2. Export any existing live data from Base44 and import into Supabase
3. Write a thin adapter module (`src/api/db.ts`) that exposes the same interface as `base44.entities.X.{list, create, update, delete}` — this allows each of the 26 call sites to be migrated one file at a time without a big-bang cutover
4. Replace `base44Client.js` with the Supabase JS client initialisation

Phase 2 — Authentication

Estimated effort: 3–5 days

1. Replace `AuthContext.jsx` — swap `base44.auth.me()` , `base44.auth.logout()` , and `base44.auth.redirectToLogin()` for `supabase.auth.getUser()` , `supabase.auth.signOut()` , and the Supabase redirect flow
2. Map Supabase's user metadata (`user_metadata`) to the existing `role` / `managed_team_ids` model — these can be stored in the `User` table as now, joined on `auth.uid()`
3. Replace the `ensureUserExists` function logic with a Supabase **Auth Trigger** (PostgreSQL function that fires on `auth.users` insert)
4. Re-implement the user invitation flow using `supabase.auth.admin.inviteUserByEmail()` and the `PendingUserTeams` table

Phase 3 — Server-Side Functions

Estimated effort: 3–5 days

1. Port all 9 Deno functions to **Supabase Edge Functions** (same Deno runtime — largely copy/paste)
2. Key changes per function:
   - Replace `base44.asServiceRole.entities.*` calls with the Supabase service-role client (`createClient(url, serviceKey)`)
   - Replace `base44.users.inviteUser()` with `supabase.auth.admin.inviteUserByEmail()`
   - Jira functions: no runtime changes needed — they use `fetch()` directly against the Jira API
3. Store Jira credentials (`JIRA_BASE_URL` , `JIRA_EMAIL` , `JIRA_API_TOKEN`) as Supabase Edge Function secrets

Phase 4 — Wiring, Testing & Cutover

Estimated effort: 3–5 days

1. Remove `@base44/sdk` and `@base44/vite-plugin` from `package.json` and `vite.config.js`
2. Update all 26 files that import from `base44Client` to use the new adapter

3. End-to-end testing across all pages and workflows
4. Cutover: point the app at the production Supabase project

---

7. Items That Need Custom Re-implementation

These features exist as built-in Base44 platform capabilities and will need to be rebuilt:

Feature	Current Approach	Replacement Approach
User impersonation	`ImpersonateUserDialog` uses Base44 built-in	Store an `impersonated_user_id` in a React context; query as that user
Session timeout	Base44 session management handles expiry	Implement via `supabase.auth.onAuthStateChange` + an inactivity timer (already partially done in `Layout.jsx`)
Pending invite + team assignment	`PendingUserTeams` entity + `inviteUserWithTeams` function	Keep same pattern; use the `PendingUserTeams` table, replace the Base44 invite API with Supabase invite

---

8. Risks & Mitigations

Risk	Likelihood	Mitigation
Data loss during export/import	Low	Export to JSON, validate record counts before cutover; keep Base44 read-only for 2 weeks after migration
Auth session disruption for active users	Medium	Plan a maintenance window; all users re-login once
Jira function behaviour differences	Low	Functions are already isolated Deno scripts; run them in staging against a test Jira project first
Permissions model drift	Low	`permissions.js` is pure JS with no Base44 dependency; it stays unchanged
Supabase RLS complexity	Low	Permission checks can remain in the application layer exactly as they are today; RLS can be added later incrementally

---

9. What Does Not Change

To be explicit about scope boundaries — the following **require no migration work at all**:

- All React components (`src/components/`)
- All page logic outside of data fetching (`src/pages/`)
- `src/lib/permissions.js` — role checks are pure JavaScript
- `src/lib/quarter-utils.js` , `useQuarters.js` , and all other utility hooks
- React Router v6 routing
- React Query (TanStack Query) — all `useQuery` / `useMutation` calls stay the same
- All export functionality (Excel, CSV, PDF via `xlsx` , `jspdf` , `html2canvas`)
- All UI components, charts, and styling

---

10. Summary & Recommendation

Total files to touch	~40 (26 call sites + 9 functions + 4–5 infrastructure files)
Estimated total effort	3–4 weeks for a focused engineer
Recommended target	Supabase (open source, Deno Edge Functions, PostgreSQL)
Risk level	Low–Medium — UI is fully decoupled; risk is in auth and data migration
Reversibility	High during Phase 1–2; after cutover, keep Base44 read-only for a buffer period

> **Recommendation:** Proceed with Supabase. Start with Phase 1 (schema + adapter) while the app continues to run on Base44, so the migration can be tested in parallel without any downtime or user impact.